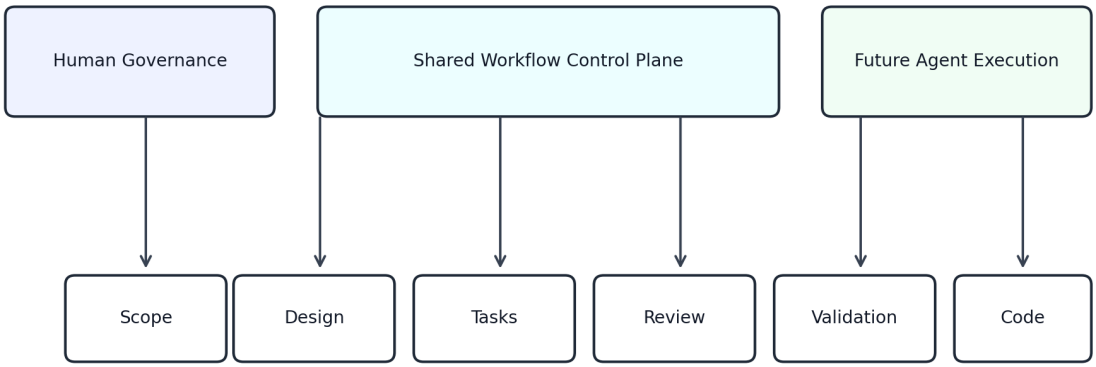
Enterprise Workflow Architecture

Professional manual for a workflow-first engineering operating model

This version focuses on why the system exists, how shared workflow architecture is structured, how projects consume it safely, and how future agents plug into the model without redefining governance.

High-level approach: humans govern, workflow stabilizes, agents plug in later



Design principle: humans govern and review; workflows stay deterministic; agents plug in later for implementation work.

Contents

1	High-level design approach and motivation
2	Executive summary
3	Shared root architecture
4	Project workspace model
5	Lifecycle and governance
6	Skill system
7	Environment and authentication model
8	Agent integration model
9	Operations and upgrade model

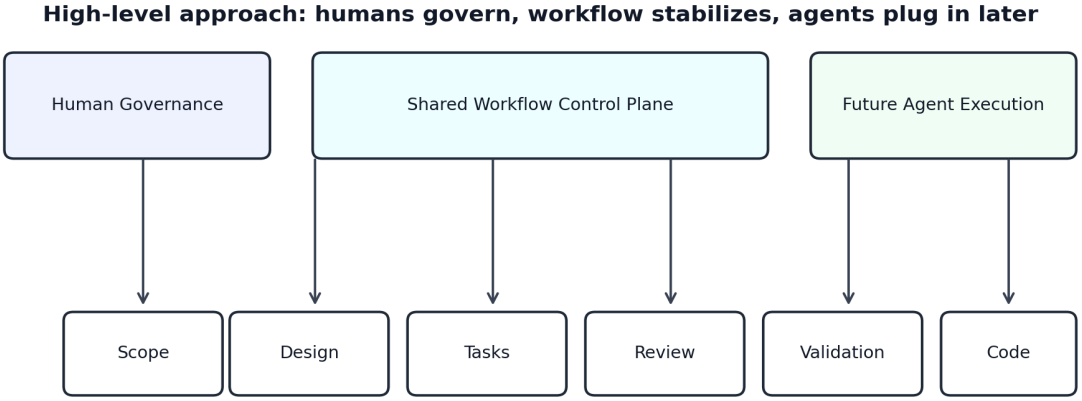
1. High-level design approach and motivation

• Modern engineering teams want higher implementation throughput without giving up review quality, architecture discipline, and release safety.

• The central motivation behind this workflow is to let humans stay responsible for judgment-heavy work while allowing agents to participate later in implementation-heavy work.

• In practice this means humans define scope, humans approve design, humans review outcomes, and implementation work can later be delegated to agents when the workflow is ready.

• This is a workflow-first model: stabilize the operating system first, then let agents plug into it.



2. Executive summary

• The current enterprise design is built around one shared canonical root at /.claude/.

• This shared root contains shared workflow rules, shared workflow skills, shared templates, shared schemas, and shared helper scripts.

• Each project then keeps its own CLAUDE.md, workspace.yaml, .env, and feature artifacts, plus a real .claude/skills directory containing per-skill symlinks.

• This fixes earlier issues such as drifting templates, whole-directory skill symlinks, monolithic tasks files, and incorrect assumptions about SSH being sufficient for PR creation.

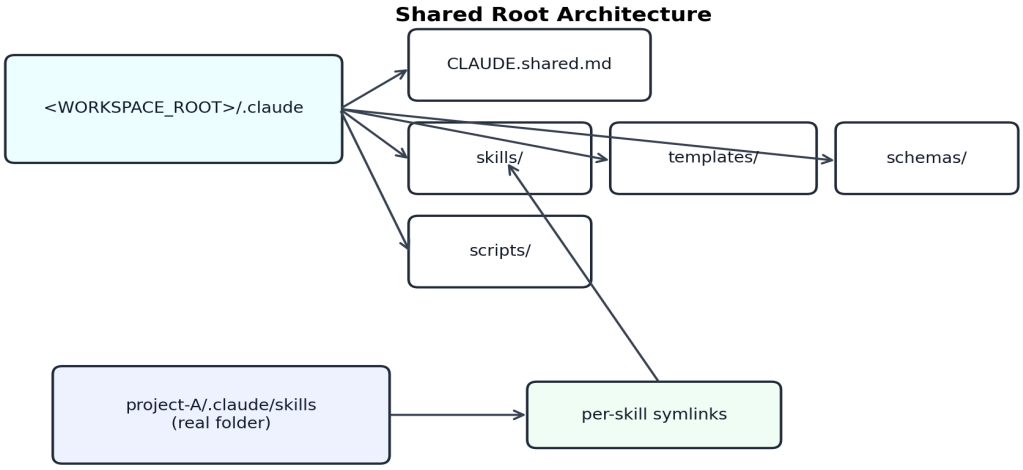
3. Shared root architecture

• A single shared root keeps company workflow rules centralized and makes upgrades practical.

• Without this root, workflow rules drift between projects and templates slowly diverge.

• The shared root should contain CLAUDE.shared.md, skills, templates, schemas, and scripts.

• Projects consume the shared root rather than redefining the company model from scratch.



4. Project workspace model

• Each project workspace keeps its own CLAUDE.md, workspace.yaml, .env, docs, and feature artifacts.

• Project CLAUDE.md should be structured into project-local context, a synced shared rules section, and project-specific additional rules.

• The project .claude/skills folder must remain a real directory, not a whole-directory symlink.

• Shared skills are linked one by one so local project-only skills remain possible.

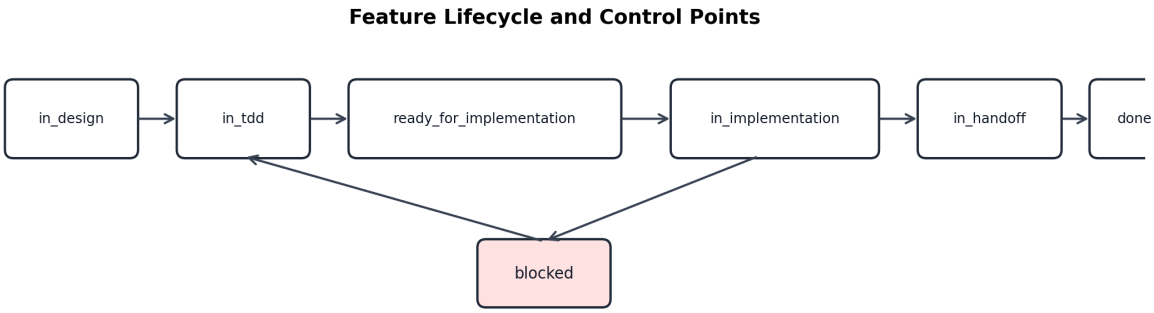
5. Lifecycle and governance

• Features progress through design, technical design, implementation readiness, implementation, handoff, and completion.

• Tasks progress through todo, ready, in progress, in review, and done, with blocked and cancelled states when needed.

• The standard workflow remains: spec, design, tasks, implementation, handoff, approval.

• A task should become done only after human review and confirmation. Agents may move work to in_review, but not to done.



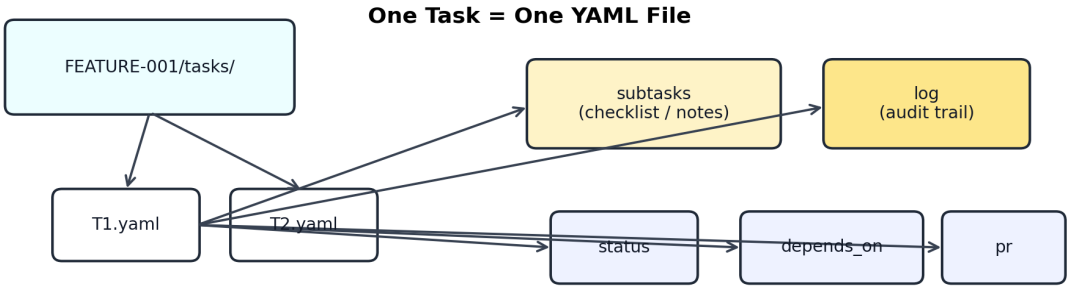
6. Skill system

• The shared workflow skill set includes init-workspace, init-feature, sync-workspace-rules, plan-first, approve-feature, reject-feature, set-feature-stage, start-implementation, resolve-project-env, pr-self-review, pr-create, and generate-deployment-checklist.

• Bootstrap happens first through install.sh. After bootstrap, projects use shared skills through per-skill symlinks.

• init-workspace is responsible for creating the workspace and invoking install.sh.

• sync-workspace-rules is responsible for syncing shared rules and repairing per-skill symlinks.



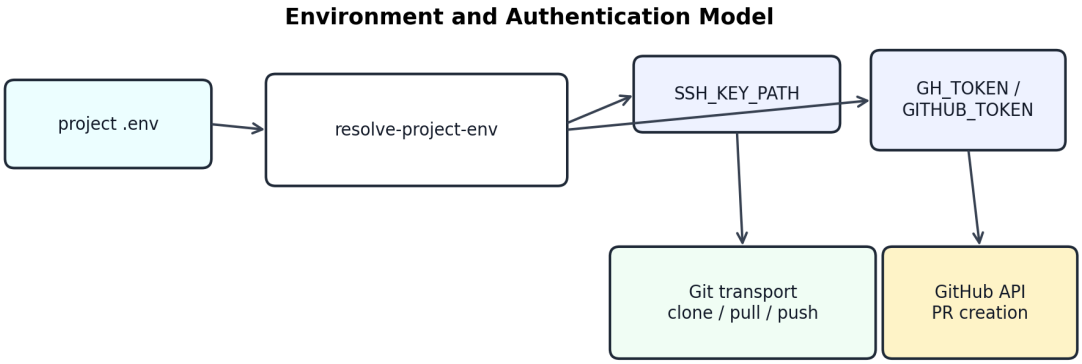
7. Environment and authentication model

• Workflow-relevant values must come from project .env rather than assumptions.

• Two authentication modes are modeled explicitly: SSH_KEY_PATH for git transport, and GH_TOKEN / GITHUB_TOKEN for GitHub API operations.

• SSH alone is not sufficient for automatic PR creation.

• If no API token is available, pr-create must fall back to a manual PR URL and record that honestly.



8. Agent integration model

• This package is workflow-first and does not define agent runtime implementation yet.

• Agents are treated as future external workers that will join the workflow later.

• Later, agents may read workspace.yaml, task YAML files, status files, and project .env via resolve-project-env.

• Later, agents may move tasks from ready to in_progress and from in_progress to in_review, but not approve stages or mark work done.

9. Operations and upgrade model

• Recommended bootstrap sequence: install shared root, create or upgrade project workspace, run install.sh, sync shared workflow rules, then begin normal operation.

• install.sh is the bootstrap path because project-level shared skills cannot repair themselves before the project has been wired to the shared root.

• Workflow evolution should be additive: new sections, migration notes, and upgrade packs are preferred over rewriting existing semantics.

• Common failure modes now addressed include whole-directory skill symlinks, missing WORKSPACE_ROOT, missing SSH_KEY_PATH, and assuming SSH is enough for PR creation.

Appendix — Final Workspace .env.template

```
# Copy this file to `.env` and fill in values for your local machine / identity.
# Do not commit real secrets if this workspace is shared.

# Canonical local root that contains:
# - .claude/
# - project folders
WORKSPACE_ROOT=/Users/you/workspace

# Workspace identity
WORKSPACE_ID=project-A
WORKSPACE_NAME=Project A

# Git identity used by workflow approvals / audit trail
GIT_AUTHOR_NAME=
GIT_AUTHOR_EMAIL=

# GitHub / repo access
GITHUB_ACCOUNT=
SSH_KEY_PATH=~/.ssh/your_key_name

# GitHub API access for PR creation
GH_TOKEN=
GITHUB_TOKEN=

# Optional local preferences
DEFAULT_BASE_BRANCH=main
SKIP_STAGING=false
```